



Université Hassan II de Casablanca
Faculté des Sciences & Techniques de Mohammed VI

Licence Informatique, Réseaux & Multimédia

Collecte et Traitement de Données Multimodales

Bibliothèques Python pour les modalités
Audio et Image



Guide pratique, progressif et orienté traitement multimodal
de la représentation numérique à l'extraction de caractéristiques

Réalisé par :
Prof. Abdellah ADIB

Août 2026

Contents

1	Introduction	2
1.1	Contexte et Objectifs	2
1.2	Périmètre du Rapport	2
1.3	Installation des Bibliothèques	3
2	NumPy : l'Essentiel pour les Données Multimodales	4
2.1	Pourquoi NumPy ?	4
2.2	Importer NumPy	4
2.3	Le Tableau NumPy : <code>ndarray</code>	4
2.3.1	Tableau à une Dimension	4
2.3.2	Tableau à Deux Dimensions	5
2.3.3	Tableau à Trois Dimensions	5
2.4	Les Propriétés Essentielles d'un Tableau	5
2.4.1	Pourquoi le <code>dtype</code> est Important ?	6
2.5	Créer des Tableaux Utiles	6
2.5.1	<code>np.zeros()</code> et <code>np.ones()</code>	6
2.5.2	<code>np.arange()</code>	6
2.5.3	<code>np.linspace()</code>	6
2.6	Accéder aux Valeurs	7
2.6.1	Indexation d'un Signal	7
2.6.2	Indexation d'une Matrice	7
2.7	Les Tranches : <i>Slicing</i>	7
2.7.1	Extraire une Partie d'un Signal	7
2.7.2	Extraire une Région d'une Image	8
2.7.3	Accéder à un Canal	8
2.8	Copier un Tableau	8
2.9	Changer le Type Numérique	8
2.10	Opérations Élément par Élément	8
2.11	Statistiques Élémentaires	9
2.12	Masques Booléens : Notion de Base	9
2.13	Restructurer avec <code>reshape()</code>	9
2.14	Concaténer des Tableaux	10
2.15	Construire un Axe Temporel pour l'Audio	10
2.16	Ce qu'il Faut Maîtriser avant de Continuer	10
3	Matplotlib : l'Essentiel pour Visualiser les Données	11
3.1	Pourquoi Matplotlib ?	11
3.2	Importer Matplotlib	11
3.3	Premier Tracé	11
3.4	Figure et Axes	11
3.5	Tracer un Signal Audio	12
3.5.1	Ajouter une Grille	12

3.6	Afficher une Image avec <code>imshow()</code>	12
3.6.1	Image en Niveaux de Gris	12
3.7	Attention à l'Ordre des Couleurs	13
3.8	Afficher Plusieurs Graphiques	13
3.9	Titres, Axes et Légende	13
3.10	Limiter les Axes	14
3.11	Sauvegarder une Figure	14
3.12	Exemple Complet : Signal	14
3.13	Exemple Complet : Image	15
3.14	Ce qu'il Faut Maîtriser avant de Continuer	15
3.15	Lien avec les Chapitres Suivants	16
4	OpenCV : Prise en Main de la Modalité Image	17
4.1	Principe Fondamental	17
4.1.1	Importer les Bibliothèques	17
4.2	Ouvrir une Image	17
4.3	Représenter une Image en Mémoire	18
4.3.1	Le Tableau NumPy	18
4.3.2	Ordre BGR	18
4.4	Afficher une Image	18
4.4.1	Affichage avec OpenCV	18
4.4.2	Affichage avec Matplotlib	18
4.4.3	Afficher une Image en Niveaux de Gris	19
4.5	Manipulations Élémentaires	19
4.5.1	Accéder à un Pixel	19
4.5.2	Modifier un Pixel	19
4.5.3	Sélectionner une Région d'Intérêt	19
4.5.4	Redimensionner une Image	19
4.5.5	Retourner une Image	19
4.6	Sauvegarder une Image	20
4.7	Exemple Complet	20
5	Librosa : Représentation Numérique de la Modalité Audio	21
5.1	Principe Fondamental	21
5.2	Ouvrir un Fichier Audio	21
5.3	Représenter le Signal en Mémoire	21
5.3.1	Nombre d'Échantillons et Durée	22
5.3.2	Construire l'Axe Temporel	22
5.4	Afficher la Forme d'Onde	22
5.4.1	Avec Librosa	22
5.4.2	Avec Matplotlib	22
5.5	Manipulations Élémentaires	22
5.5.1	Extraire un Intervalle Temporel	23
5.5.2	Accéder aux Échantillons	23
5.5.3	Modifier Simplement l'Amplitude	23
5.6	Écouter dans un Notebook	23
5.7	Librosa et la Sauvegarde	23
5.8	Exemple Complet	24
6	Pydub : Manipulation Simple des Fichiers Audio	25
6.1	Principe Fondamental	25
6.2	Ouvrir un Fichier Audio	25

6.3	Représenter un AudioSegment	26
6.4	Écouter un AudioSegment	26
6.5	Manipulations Élémentaires	26
6.5.1	Découper un Segment	26
6.5.2	Concaténer Deux Segments	26
6.5.3	Répéter un Segment	26
6.5.4	Modifier le Niveau Sonore	26
6.5.5	Changer le Nombre de Canaux	26
6.5.6	Changer la Fréquence d'Échantillonnage	27
6.6	Sauvegarder et Convertir	27
6.7	Exemple Complet	27
7	Complémentarité des Bibliothèques	28
7.1	OpenCV pour l'Image	28
7.2	Librosa et Pydub pour l'Audio	28
7.3	Rôle de NumPy et Matplotlib	28
7.4	Arbre de Décision	29
8	Conclusion et Bonnes Pratiques	30
8.1	Récapitulatif	30
8.2	Bonnes Pratiques Générales	30
8.3	Ce Qui Sera Traité Ultérieurement	30
9	Aide-Mémoire NumPy et Matplotlib	32
9.1	NumPy	32
9.2	Matplotlib	33
10	Bibliographie et Ressources	34
10.1	Documentation Officielle	34
10.2	Fonctions à Maîtriser dans ce Rapport	34

Chapter 1

Introduction

1.1 Contexte et Objectifs

Avant d'appliquer des traitements avancés à une image ou à un signal audio, il est indispensable de savoir **prendre possession de la donnée** dans un programme Python. Cette étape élémentaire consiste à comprendre comment un fichier multimédia est ouvert, sous quelle forme son contenu est représenté en mémoire, comment l'afficher ou l'écouter, comment réaliser quelques manipulations de base et comment enregistrer le résultat.

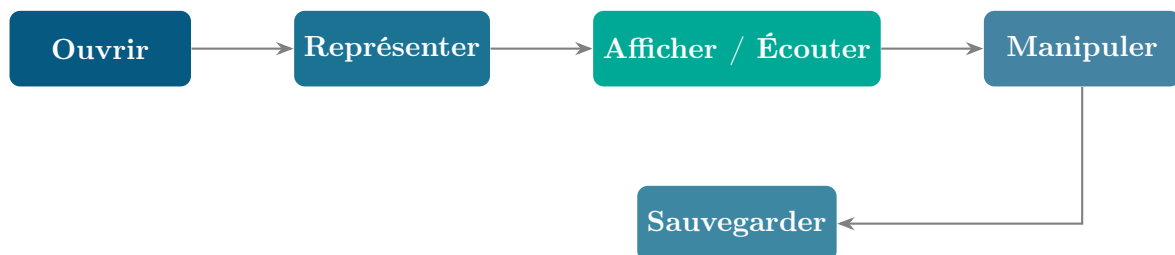
Ce rapport introduit cinq outils complémentaires, avec un niveau volontairement adapté à la prise en main des données multimodales :

- **NumPy** pour comprendre et manipuler les tableaux numériques qui représentent les données ;
- **Matplotlib** pour visualiser ces données dans un environnement scientifique ;
- **OpenCV** pour la modalité image ;
- **Librosa** pour représenter un signal audio sous forme numérique ;
- **Pydub** pour manipuler simplement des fichiers et des segments audio.

NumPy et Matplotlib sont présentés en premier car ils constituent le socle commun aux chapitres consacrés à l'image et à l'audio. Le but n'est toutefois pas d'en faire un cours exhaustif : seules les notions indispensables à la suite sont retenues.

1.2 Périmètre du Rapport

La progression retenue est volontairement limitée à :



Les traitements avancés ne sont donc **pas** abordés ici. Le filtrage, la segmentation, la détection de contours, les transformations fréquentielles, les spectrogrammes, les MFCC et l'extraction de

caractéristiques feront l'objet de rapports dédiés.

1.3 Installation des Bibliothèques

```
1 pip install numpy matplotlib opencv-python librosa pydub
```

Pour Pydub, certains formats compressés (MP3, AAC, etc.) nécessitent en pratique la présence d'un outil externe tel que FFmpeg. Le format WAV permet de commencer sans multiplier les dépendances.

Licence IPRM

Chapter 2

NumPy : l'Essentiel pour les Données Multimodales

2.1 Pourquoi NumPy ?

NumPy est la bibliothèque de référence pour la manipulation de tableaux numériques en Python. Dans ce rapport, son rôle est fondamental : une image ouverte avec OpenCV est représentée par un tableau NumPy, et un signal chargé avec Librosa est également représenté par un tableau NumPy.

L'objectif de ce chapitre est donc de maîtriser uniquement les mécanismes nécessaires pour **comprendre, inspecter et manipuler les données image et audio**.

2.2 Importer NumPy

La convention habituelle consiste à importer NumPy sous l'alias `np` :

```
1 import numpy as np
```

2.3 Le Tableau NumPy : ndarray

L'objet central de NumPy est le tableau multidimensionnel `ndarray`.

```
1 import numpy as np
2
3 signal = np.array([0.2, 0.5, -0.3, 0.1])
4
5 print(signal)
6 print(type(signal))
```

Contrairement à une simple liste Python, un tableau NumPy est conçu pour effectuer efficacement des opérations numériques sur un grand nombre de valeurs.

2.3.1 Tableau à une Dimension

Un signal audio mono peut être représenté par une suite d'échantillons :

```
1 signal = np.array([
2     0.10,
3     0.25,
4     -0.15,
5     0.30,
6     0.05
7 ])
8
9 print(signal.shape)
```

La forme obtenue est :

(5,)

Il s'agit d'un tableau à une dimension contenant cinq valeurs.

2.3.2 Tableau à Deux Dimensions

Une image en niveaux de gris peut être vue comme une matrice :

```
1 image_gris = np.array([
2     [0, 50, 100],
3     [150, 200, 255]
4 ])
5
6 print(image_gris.shape)
```

La forme (2, 3) signifie : 2 lignes et 3 colonnes.

2.3.3 Tableau à Trois Dimensions

Une image couleur possède généralement trois composantes par pixel :

```
1 image_couleur = np.zeros(
2     (480, 640, 3),
3     dtype=np.uint8
4 )
5
6 print(image_couleur.shape)
```

La forme (480, 640, 3) correspond ici à :

- 480 lignes, donc la hauteur ;
- 640 colonnes, donc la largeur ;
- 3 canaux de couleur.

2.4 Les Propriétés Essentielles d'un Tableau

Les propriétés suivantes doivent devenir des réflexes :

```
1 a = np.array([
2     [10, 20, 30],
3     [40, 50, 60]
4 ])
```



```

5
6 print("Forme :", a.shape)
7 print("Dimensions :", a.ndim)
8 print("Nombre de valeurs :", a.size)
9 print("Type :", a.dtype)

```

Propriété	Signification
shape	taille du tableau suivant chacun de ses axes
ndim	nombre de dimensions du tableau
size	nombre total de valeurs contenues dans le tableau
dtype	type numérique utilisé pour stocker les valeurs

2.4.1 Pourquoi le dtype est Important ?

Une image classique est souvent stockée avec le type `uint8`, dont les valeurs vont de 0 à 255. Un signal audio chargé par Librosa est généralement représenté avec un type flottant.

```

1 pixels = np.array(
2     [0, 127, 255],
3     dtype=np.uint8
4 )
5
6 signal = np.array(
7     [0.2, -0.5, 0.8],
8     dtype=np.float32
9 )
10
11 print(pixels.dtype)
12 print(signal.dtype)

```

2.5 Créer des Tableaux Utiles

2.5.1 `np.zeros()` et `np.ones()`

```

1 a = np.zeros((3, 4))
2 b = np.ones((2, 5))
3
4 print(a)
5 print(b)

```

2.5.2 `np.arange()`

```

1 indices = np.arange(10)
2 print(indices)

```

Cette fonction sera particulièrement utile pour construire l'axe temporel d'un signal audio.

2.5.3 `np.linspace()`

```

1 temps = np.linspace(

```

```
2     0,  
3     1,  
4     1000,  
5     endpoint=False  
6 )  
7  
8 print(temps.shape)
```

Elle crée ici 1000 valeurs régulièrement réparties entre 0 et 1 seconde.

2.6 Accéder aux Valeurs

2.6.1 Indexation d'un Signal

```
1 signal = np.array([  
2     0.1,  
3     0.2,  
4     -0.3,  
5     0.4,  
6     0.5  
7 ])  
8  
9 print(signal[0])  
10 print(signal[2])  
11 print(signal[-1])
```

2.6.2 Indexation d'une Matrice

```
1 a = np.array([  
2     [10, 20, 30],  
3     [40, 50, 60]  
4 ])  
5  
6 print(a[0, 0])  
7 print(a[1, 2])
```

Dans une image, cette logique devient :

```
1 pixel = image[100, 200]
```

L'indice 100 désigne la ligne et l'indice 200 la colonne.

2.7 Les Tranches : *Slicing*

Le slicing permet de sélectionner une partie d'un tableau sans écrire de boucle.

2.7.1 Extraire une Partie d'un Signal

```
1 signal = np.arange(20)  
2  
3 debut = signal[:5]  
4 milieu = signal[5:10]
```

```
5 un_sur_deux = signal[:,2]
6
7 print(debut)
8 print(milieu)
9 print(un_sur_deux)
```

2.7.2 Extraire une Région d'une Image

```
1 region = image[100:300, 200:500]
```

Cette seule instruction sélectionne les lignes 100 à 299 et les colonnes 200 à 499.

2.7.3 Accéder à un Canal

Pour une image couleur de forme (H, W, 3) :

```
1 canal_0 = image[:, :, 0]
2 canal_1 = image[:, :, 1]
3 canal_2 = image[:, :, 2]
```

Le symbole `:` signifie ici : prendre toutes les valeurs de l'axe concerné.

2.8 Copier un Tableau

```
1 copie = image.copy()
```

Cette instruction est importante lorsqu'on souhaite modifier une donnée tout en conservant l'original.

2.9 Changer le Type Numérique

```
1 image_float = image.astype(np.float32)
2
3 print(image.dtype)
4 print(image_float.dtype)
```

On peut ensuite, par exemple, passer d'une échelle 0-255 à une échelle 0-1 :

```
1 image_float = image.astype(
2     np.float32
3 ) / 255.0
```

La signification et l'intérêt des différentes normalisations seront étudiés dans les rapports de traitement dédiés.

2.10 Opérations Élément par Élément

```
1 signal = np.array([
2     0.2,
```

```
3     -0.4,
4     0.6
5 ])
6
7 signal_reduit = 0.5 * signal
8 signal_inverse = -signal
9
10 print(signal_reduit)
11 print(signal_inverse)
```

NumPy applique automatiquement l'opération à toutes les valeurs du tableau.

2.11 Statistiques Élémentaires

```
1 print("Minimum :", signal.min())
2 print("Maximum :", signal.max())
3 print("Moyenne :", signal.mean())
4 print("Ecart-type :", signal.std())
5 print("Somme :", signal.sum())
```

Ces fonctions servent ici essentiellement à **inspecter** les données. Leur interprétation statistique approfondie sera abordée dans les chapitres où elle devient nécessaire.

2.12 Masques Booléens : Notion de Base

Une condition appliquée à un tableau produit un tableau de valeurs booléennes.

```
1 x = np.array([
2     -2,
3     -1,
4     0,
5     1,
6     2
7 ])
8
9 masque = x > 0
10
11 print(masque)
12 print(x[masque])
```

Cette notion est utile pour sélectionner des valeurs répondant à une condition.

2.13 Restructurer avec reshape()

```
1 a = np.arange(12)
2
3 matrice = a.reshape(3, 4)
4
5 print(matrice)
6 print(matrice.shape)
```

Le nombre total de valeurs doit rester identique.

2.14 Concaténer des Tableaux

Pour l'audio, on peut assembler deux portions de signal :

```
1 a = np.array([0.1, 0.2, 0.3])
2 b = np.array([0.4, 0.5])
3
4 c = np.concatenate([a, b])
5
6 print(c)
```

2.15 Construire un Axe Temporel pour l'Audio

Si un signal contient N échantillons et possède une fréquence d'échantillonnage sr :

```
1 temps = np.arange(len(signal)) / sr
```

Ainsi :

- `signal[i]` contient l'amplitude ;
- `temps[i]` contient l'instant correspondant en secondes.

2.16 Ce qu'il Faut Maîtriser avant de Continuer

À ce stade, l'étudiant doit savoir :

- reconnaître un `ndarray` ;
- expliquer `shape`, `ndim`, `size` et `dtype` ;
- accéder à une valeur ;
- sélectionner une tranche ;
- extraire une région d'image ;
- copier un tableau ;
- convertir son type ;
- effectuer une opération élément par élément ;
- calculer quelques statistiques simples ;
- construire un axe temporel.

Chapter 3

Matplotlib : l'Essentiel pour Visualiser les Données

3.1 Pourquoi Matplotlib ?

Matplotlib est une bibliothèque de visualisation scientifique. Dans ce rapport, elle est utilisée pour répondre à deux besoins simples mais fondamentaux :

- afficher une **courbe**, notamment la forme temporelle d'un signal audio ;
- afficher une **image** représentée par un tableau NumPy.

L'objectif n'est pas d'étudier toutes les possibilités graphiques de Matplotlib, mais d'acquérir les réflexes nécessaires pour contrôler visuellement les données manipulées.

3.2 Importer Matplotlib

La convention habituelle est :

```
1 import matplotlib.pyplot as plt
```

3.3 Premier Tracé

```
1 x = np.array([0, 1, 2, 3, 4])
2 y = np.array([0, 1, 4, 9, 16])
3
4 plt.plot(x, y)
5 plt.show()
```

`plt.show()` demande l'affichage de la figure.

3.4 Figure et Axes

Pour des programmes structurés, on utilisera de préférence :

```
1 fig, ax = plt.subplots()
2
```

```
3 ax.plot(x, y)
4
5 plt.show()
```

Deux objets apparaissent :

- `fig` : la figure complète ;
- `ax` : la zone dans laquelle les données sont tracées.

Cette écriture sera privilégiée dans la suite du cours car elle facilite la construction de figures comportant plusieurs graphiques.

3.5 Tracer un Signal Audio

```
1 temps = np.arange(len(signal)) / sr
2
3 fig, ax = plt.subplots()
4
5 ax.plot(temps, signal)
6
7 ax.set_title("Signal audio")
8 ax.set_xlabel("Temps (s)")
9 ax.set_ylabel("Amplitude")
10
11 plt.show()
```

3.5.1 Ajouter une Grille

```
1 ax.grid(True)
```

La grille peut faciliter la lecture d'une courbe, mais elle doit rester un outil et non un élément décoratif.

3.6 Afficher une Image avec `imshow()`

```
1 fig, ax = plt.subplots()
2
3 ax.imshow(image_rgb)
4
5 ax.set_title("Image RGB")
6 ax.axis("off")
7
8 plt.show()
```

`imshow()` interprète les valeurs du tableau comme des pixels.

3.6.1 Image en Niveaux de Gris

```
1 fig, ax = plt.subplots()
2
3 ax.imshow(
```

```
4     image_gris ,
5     cmap="gray"
6 )
7
8 ax.axis("off")
9
10 plt.show()
```

Le paramètre `cmap="gray"` indique que les valeurs doivent être représentées avec une palette de niveaux de gris.

3.7 Attention à l'Ordre des Couleurs

OpenCV charge une image couleur dans l'ordre BGR alors que Matplotlib affiche normalement une image couleur dans l'ordre RGB.

Il faut donc convertir l'image avant l'affichage :

```
1 image_rgb = cv2.cvtColor(
2     image,
3     cv2.COLOR_BGR2RGB
4 )
5
6 plt.imshow(image_rgb)
7 plt.show()
```

Ce point sera repris dans le chapitre consacré à OpenCV.

3.8 Afficher Plusieurs Graphiques

```
1 fig, axes = plt.subplots(
2     1,
3     2,
4     figsize=(10, 4)
5 )
6
7 axes[0].plot(temps, signal)
8 axes[0].set_title("Signal")
9
10 axes[1].imshow(image_rgb)
11 axes[1].set_title("Image")
12 axes[1].axis("off")
13
14 plt.tight_layout()
15 plt.show()
```

Cette structure permet de comparer plusieurs représentations dans une même figure.

3.9 Titres, Axes et Légende

```
1 fig, ax = plt.subplots()
2
3 ax.plot(
```



```
4     temps ,
5     signal ,
6     label="Signal original"
7 )
8
9 ax.set_title("Forme d'onde")
10 ax.set_xlabel("Temps (s)")
11 ax.set_ylabel("Amplitude")
12 ax.legend()
13
14 plt.show()
```

Une figure destinée à un rapport doit être compréhensible sans devoir lire le code qui l'a produite.

3.10 Limiter les Axes

```
1 ax.set_xlim(0, 2)
2 ax.set_ylim(-1, 1)
```

Ces commandes permettent de contrôler la zone visible. Elles ne modifient pas les données.

3.11 Sauvegarder une Figure

```
1 fig.savefig(
2     "resultats/figure.png",
3     dpi=200,
4     bbox_inches="tight"
5 )
```

Il est préférable de sauvegarder la figure à partir de l'objet `fig`.

3.12 Exemple Complet : Signal

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 sr = 1000
5 duree = 1.0
6
7 temps = np.arange(
8     int(sr * duree)
9 ) / sr
10
11 signal = np.sin(
12     2 * np.pi * 5 * temps
13 )
14
15 fig, ax = plt.subplots(
16     figsize=(8, 3)
17 )
18
19 ax.plot(temps, signal)
```

```
20
21 ax.set_title("Signal sinusoidal")
22 ax.set_xlabel("Temps (s)")
23 ax.set_ylabel("Amplitude")
24 ax.grid(True)
25
26 fig.tight_layout()
27 plt.show()
```

Cet exemple sert uniquement à illustrer la représentation d'un signal. L'étude des sinusoides, fréquences et traitements spectraux appartient au rapport consacré au traitement audio.

3.13 Exemple Complet : Image

```
1 import cv2
2 import matplotlib.pyplot as plt
3
4 image = cv2.imread(
5     "images/photo.jpg"
6 )
7
8 image_rgb = cv2.cvtColor(
9     image,
10    cv2.COLOR_BGR2RGB
11 )
12
13 fig, ax = plt.subplots()
14
15 ax.imshow(image_rgb)
16 ax.set_title("Photographie")
17 ax.axis("off")
18
19 fig.tight_layout()
20 plt.show()
```

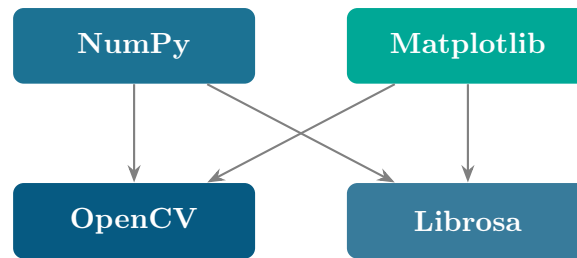
3.14 Ce qu'il Faut Maîtriser avant de Continuer

À ce stade, l'étudiant doit savoir :

- importer `matplotlib.pyplot` ;
- créer une figure avec `plt.subplots()` ;
- tracer une courbe avec `ax.plot()` ;
- afficher une image avec `ax.imshow()` ;
- utiliser `cmap="gray"` ;
- ajouter un titre et nommer les axes ;
- masquer les axes d'une image ;
- créer plusieurs zones de tracé avec `subplots()` ;
- sauvegarder une figure avec `fig.savefig()`.

3.15 Lien avec les Chapitres Suivants

NumPy et Matplotlib constituent maintenant le socle nécessaire pour comprendre les bibliothèques spécialisées :



OpenCV et Librosa pourront ainsi être étudiés en se concentrant sur leur rôle propre, sans devoir réintroduire à chaque fois la représentation par tableaux ou les mécanismes élémentaires d’affichage.

Chapter 4

OpenCV : Prise en Main de la Modalité Image



Figure 4.1: Bibliothèques utilisées pour la prise en main d'une image

4.1 Principe Fondamental

Une image numérique peut être vue comme une grille de pixels. Lorsqu'OpenCV ouvre un fichier image, il décode ce fichier et fournit en Python un tableau NumPy. On peut alors inspecter sa taille, accéder à ses pixels, sélectionner une région, modifier des valeurs et sauvegarder le résultat.

4.1.1 Importer les Bibliothèques

```
1 import cv2
2 import numpy as np
3 import matplotlib.pyplot as plt
```

4.2 Ouvrir une Image

La fonction de base est `cv2.imread()`.

```
1 image = cv2.imread("images/photo.jpg")
2
3 if image is None:
4     raise FileNotFoundError("Image introuvable ou illisible")
```

Les modes de lecture les plus utiles à ce niveau sont :

- `cv2.IMREAD_COLOR` : lecture couleur, mode par défaut ;
- `cv2.IMREAD_GRAYSCALE` : lecture directe en niveaux de gris ;

- `cv2.IMREAD_UNCHANGED` : conservation des canaux disponibles, notamment le canal alpha.

4.3 Représenter une Image en Mémoire

4.3.1 Le Tableau NumPy

```
1 print(type(image))
2 print(image.shape)
3 print(image.dtype)
4 print(image.size)
```

Pour une image couleur classique :

- `image.shape[0]` : hauteur ;
- `image.shape[1]` : largeur ;
- `image.shape[2]` : nombre de canaux ;
- `image.dtype` : type des valeurs, souvent `uint8`.

```
1 hauteur, largeur, canaux = image.shape
2 print("Hauteur :", hauteur)
3 print("Largeur :", largeur)
4 print("Canaux :", canaux)
```

4.3.2 Ordre BGR

OpenCV représente par défaut une image couleur selon l'ordre **BGR** : bleu, vert, rouge. Cette convention est importante lorsque l'image est transmise à Matplotlib, qui attend usuellement l'ordre RGB.

4.4 Afficher une Image

4.4.1 Affichage avec OpenCV

```
1 cv2.imshow("Image", image)
2 cv2.waitKey(0)
3 cv2.destroyAllWindows()
```

Cette méthode ouvre une fenêtre graphique et convient surtout à l'exécution locale d'un script.

4.4.2 Affichage avec Matplotlib

```
1 image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
2
3 plt.imshow(image_rgb)
4 plt.title("Image")
5 plt.axis("off")
6 plt.show()
```

4.4.3 Afficher une Image en Niveaux de Gris

```
1 gris = cv2.imread(  
2     "images/photo.jpg",  
3     cv2.IMREAD_GRAYSCALE  
4 )  
5  
6 plt.imshow(gris, cmap="gray")  
7 plt.axis("off")  
8 plt.show()
```

La conversion en niveaux de gris n'est utilisée ici que pour illustrer une autre **représentation** de l'image ; son exploitation pour des traitements sera étudiée ultérieurement.

4.5 Manipulations Élémentaires

4.5.1 Accéder à un Pixel

```
1 pixel = image[100, 200]  
2 print(pixel) # [B, G, R]
```

4.5.2 Modifier un Pixel

```
1 image_modifiee = image.copy()  
2 image_modifiee[100, 200] = [0, 0, 255]
```

4.5.3 Sélectionner une Région d'Intérêt

Une région rectangulaire s'obtient directement par les tranches NumPy :

```
1 region = image[100:300, 200:500]  
2  
3 plt.imshow(cv2.cvtColor(region, cv2.COLOR_BGR2RGB))  
4 plt.axis("off")  
5 plt.show()
```

4.5.4 Redimensionner une Image

```
1 image_redimensionnee = cv2.resize(  
2     image,  
3     (640, 480)  
4 )
```

À ce niveau, le redimensionnement est présenté comme une manipulation géométrique élémentaire. Les questions de choix d'interpolation et d'impact sur l'information seront approfondies dans le rapport consacré au traitement d'image.

4.5.5 Retourner une Image

```
1 horizontale = cv2.flip(image, 1)  
2 verticale = cv2.flip(image, 0)
```

```
3 les_deux = cv2.flip(image, -1)
```

4.6 Sauvegarder une Image

```
1 succes = cv2.imwrite(  
2     "resultats/photo_modifiee.jpg",  
3     image_modifiee  
4 )  
5  
6 print("Sauvegarde :", succes)
```

L'extension du nom de fichier détermine le format d'encodage choisi par OpenCV.

4.7 Exemple Complet

```
1 import cv2  
2 import matplotlib.pyplot as plt  
3  
4 # 1. Ouvrir  
5 image = cv2.imread("images/photo.jpg")  
6 if image is None:  
7     raise FileNotFoundError("Image introuvable")  
8  
9 # 2. Représenter / inspecter  
10 print("Forme :", image.shape)  
11 print("Type :", image.dtype)  
12  
13 # 3. Afficher  
14 image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)  
15 plt.imshow(image_rgb)  
16 plt.axis("off")  
17 plt.show()  
18  
19 # 4. Manipuler  
20 region = image[100:300, 200:500]  
21 region = cv2.flip(region, 1)  
22  
23 # 5. Sauvegarder  
24 cv2.imwrite("resultats/region.jpg", region)
```

Chapter 5

Librosa : Représentation Numérique de la Modalité Audio



Figure 5.1: Bibliothèques utilisées pour représenter et afficher un signal audio

5.1 Principe Fondamental

Un fichier audio contient une représentation codée d'un signal. Librosa permet de charger ce fichier et d'obtenir principalement deux informations :

- **y** : un tableau NumPy contenant les échantillons du signal ;
- **sr** : la fréquence d'échantillonnage, exprimée en hertz.

5.2 Ouvrir un Fichier Audio

```
1 import librosa
2 import librosa.display
3 import numpy as np
4 import matplotlib.pyplot as plt
5
6 y, sr = librosa.load(
7     "audio/voix.wav",
8     sr=None,
9     mono=True
10 )
```

L'option **sr=None** conserve la fréquence d'échantillonnage native du fichier. C'est un choix particulièrement clair pour une première prise en main : aucune modification temporelle du signal n'est introduite implicitement lors du chargement.

5.3 Représenter le Signal en Mémoire


```

1 print(type(y))
2 print(y.shape)
3 print(y.dtype)
4 print("Fréquence :", sr, "Hz")

```

5.3.1 Nombre d'Échantillons et Durée

Si le signal contient N échantillons et que la fréquence d'échantillonnage vaut f_s , sa durée est :

$$T = \frac{N}{f_s}$$

```

1 n = len(y)
2 duree = n / sr
3
4 print("Echantillons :", n)
5 print("Durée :", duree, "secondes")

```

5.3.2 Construire l'Axe Temporel

```

1 temps = np.arange(len(y)) / sr

```

Chaque valeur $y[i]$ correspond ainsi à l'amplitude du signal à l'instant approximatif i / sr .

5.4 Afficher la Forme d'Onde

5.4.1 Avec Librosa

```

1 librosa.display.waveshow(y, sr=sr)
2 plt.xlabel("Temps (s)")
3 plt.ylabel("Amplitude")
4 plt.title("Forme d'onde")
5 plt.show()

```

5.4.2 Avec Matplotlib

```

1 temps = np.arange(len(y)) / sr
2
3 plt.plot(temps, y)
4 plt.xlabel("Temps (s)")
5 plt.ylabel("Amplitude")
6 plt.title("Signal audio")
7 plt.show()

```

La forme d'onde est ici une visualisation du signal dans le domaine temporel. Les représentations fréquentielles seront étudiées dans un rapport ultérieur.

5.5 Manipulations Élémentaires

5.5.1 Extraire un Intervalle Temporel

Pour sélectionner la partie comprise entre 1,5 s et 3 s :

```
1 t_debut = 1.5
2 t_fin = 3.0
3
4 i_debut = int(t_debut * sr)
5 i_fin = int(t_fin * sr)
6
7 segment = y[i_debut:i_fin]
```

5.5.2 Accéder aux Échantillons

```
1 print(y[0])
2 print(y[:10])
3 print("Amplitude min :", y.min())
4 print("Amplitude max :", y.max())
```

5.5.3 Modifier Simplement l'Amplitude

```
1 y_reduit = 0.5 * y
```

Cette opération illustre le caractère numérique du signal. L'étude rigoureuse de la normalisation, du gain, de la dynamique et de leurs conséquences sera développée dans le rapport consacré au traitement audio.

5.6 Écouter dans un Notebook

Dans Jupyter ou Colab, l'écoute peut être réalisée avec l'outil audio d'IPython :

```
1 from IPython.display import Audio
2
3 Audio(y, rate=sr)
```

5.7 Librosa et la Sauvegarde

Librosa est principalement orientée chargement et analyse. Pour l'écriture d'un tableau audio dans un fichier, on utilise généralement une bibliothèque dédiée telle que `soundfile`, ou `Pydub` lorsqu'on travaille au niveau des fichiers et segments.

```
1 import soundfile as sf
2
3 sf.write(
4     "resultats/segment.wav",
5     segment,
6     sr
7 )
```

5.8 Exemple Complet

```
1 import librosa
2 import librosa.display
3 import matplotlib.pyplot as plt
4 from IPython.display import Audio
5
6 # 1. Ouvrir
7 y, sr = librosa.load(
8     "audio/voix.wav",
9     sr=None,
10    mono=True
11 )
12
13 # 2. Représenter / inspecter
14 print("Forme :", y.shape)
15 print("Type :", y.dtype)
16 print("Fréquence :", sr)
17 print("Durée :", len(y) / sr)
18
19 # 3. Afficher
20 librosa.display.waveshow(y, sr=sr)
21 plt.show()
22
23 # 4. Manipuler : extraire les 2 premières secondes
24 segment = y[:2 * sr]
25
26 # 5. Écouter
27 Audio(segment, rate=sr)
```

Chapter 6

Pydub : Manipulation Simple des Fichiers Audio



Figure 6.1: Pydub : manipulation pratique de segments et fichiers audio

6.1 Principe Fondamental

Pydub propose une abstraction différente de Librosa. Au lieu de manipuler en premier lieu un tableau d'échantillons, l'utilisateur travaille avec un objet `AudioSegment`. Celui-ci représente un segment audio avec ses propriétés : durée, fréquence d'échantillonnage, nombre de canaux et taille des échantillons.

Cette approche est particulièrement pratique pour découper, concaténer, modifier simplement le niveau sonore et convertir des fichiers.

6.2 Ouvrir un Fichier Audio

```
1 from pydub import AudioSegment
2
3 audio = AudioSegment.from_file(
4     "audio/voix.wav"
5 )
```

Pour un WAV :

```
1 audio = AudioSegment.from_wav("audio/voix.wav")
```

Pour un MP3, lorsque FFmpeg est correctement installé :

```
1 audio = AudioSegment.from_mp3("audio/musique.mp3")
```

6.3 Représenter un AudioSegment

```
1 print("Duree (ms) :", len(audio))
2 print("Frequence :", audio.frame_rate)
3 print("Canaux :", audio.channels)
4 print("Octets/echantillon :", audio.sample_width)
```

Contrairement à Librosa, la durée utilisée pour les opérations de découpage Pydub est exprimée en **millisecondes**.

6.4 Écouter un AudioSegment

```
1 from pydub.playback import play
2
3 play(audio)
```

Dans un notebook, on peut aussi exploiter les outils d’affichage audio adaptés à l’environnement.

6.5 Manipulations Élémentaires

6.5.1 Découper un Segment

```
1 premieres_2_secondes = audio[:2000]
2 segment = audio[1500:3000]
```

6.5.2 Concaténer Deux Segments

```
1 audio1 = AudioSegment.from_file("audio/a.wav")
2 audio2 = AudioSegment.from_file("audio/b.wav")
3
4 concatene = audio1 + audio2
```

6.5.3 Répéter un Segment

```
1 repete = audio * 3
```

6.5.4 Modifier le Niveau Sonore

```
1 plus_fort = audio + 6
2 moins_fort = audio - 6
```

Les valeurs sont exprimées en décibels. Cette opération est présentée ici comme manipulation de base ; les notions de niveau, dynamique et normalisation seront approfondies séparément.

6.5.5 Changer le Nombre de Canaux

```
1 mono = audio.set_channels(1)
2 stereo = audio.set_channels(2)
```

6.5.6 Changer la Fréquence d'Échantillonnage

```
1 audio_16k = audio.set_frame_rate(16000)
```

Cette opération doit être distinguée de la simple lecture d'une propriété. Les implications du rééchantillonnage sur le signal seront étudiées dans le rapport de traitement audio.

6.6 Sauvegarder et Convertir

```
1 audio.export(  
2     "resultats/sortie.wav",  
3     format="wav"  
4 )
```

Conversion vers MP3 :

```
1 audio.export(  
2     "resultats/sortie.mp3",  
3     format="mp3"  
4 )
```

La disponibilité des formats compressés dépend de l'installation des outils externes utilisés par Pydub.

6.7 Exemple Complet

```
1 from pydub import AudioSegment  
2  
3 # 1. Ouvrir  
4 audio = AudioSegment.from_file("audio/voix.wav")  
5  
6 # 2. Représenter / inspecter  
7 print("Durée :", len(audio), "ms")  
8 print("Fréquence :", audio.frame_rate)  
9 print("Canaux :", audio.channels)  
10  
11 # 3. Manipuler  
12 segment = audio[1000:4000]  
13 segment = segment + 3  
14  
15 # 4. Sauvegarder  
16 segment.export(  
17     "resultats/extrait.wav",  
18     format="wav"  
19 )
```

Chapter 7

Complémentarité des Bibliothèques

7.1 OpenCV pour l'Image

OpenCV joue le rôle de bibliothèque principale pour la modalité image : ouverture des fichiers, accès à la représentation matricielle, affichage local, manipulations élémentaires et sauvegarde.

7.2 Librosa et Pydub pour l'Audio

Les deux bibliothèques répondent à des besoins différents :

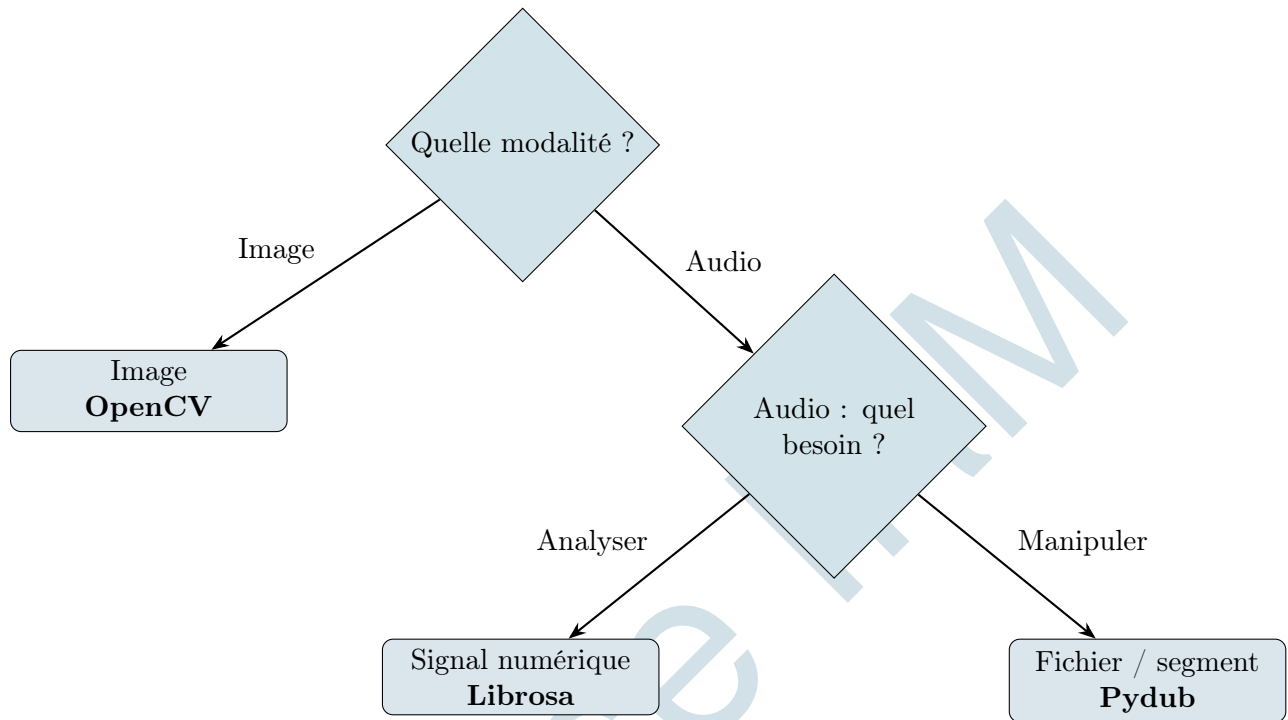
Aspect	Librosa	Pydub
Objet principal	Tableau NumPy + fréquence d'échantillonnage	AudioSegment
Orientation	Représentation numérique du signal	Manipulation pratique du fichier/segment
Temps	indices d'échantillons / secondes	millisecondes
Visualisation	forme d'onde avec Librosa/-Matplotlib	pas son objectif principal
Découpage	slicing NumPy	slicing de AudioSegment
Concaténation	possible par tableaux	très directe avec +
Conversion formats	pas son rôle principal	fonctionnalité centrale via export
Suite du cours	analyse et traitement du signal	préparation/manipulation des fichiers

7.3 Rôle de NumPy et Matplotlib

- **NumPy** fournit la structure de tableau utilisée directement par OpenCV et Librosa.
- **Matplotlib** permet de visualiser une image ou une forme d'onde dans un environnement scientifique.

Ils constituent le **socle transversal** présenté au début de ce rapport. Seul l'essentiel nécessaire à la manipulation des modalités image et audio y est étudié.

7.4 Arbre de Décision



Chapter 8

Conclusion et Bonnes Pratiques

8.1 Récapitulatif

La première compétence attendue n'est pas encore de savoir filtrer, segmenter ou extraire des caractéristiques. Elle consiste à savoir transformer un fichier multimédia en un objet Python dont on comprend précisément la structure.

- **NumPy** permet de comprendre et manipuler les tableaux numériques qui portent les données.
- **Matplotlib** permet de contrôler visuellement ces représentations.
- Pour une **image**, OpenCV fournit un tableau NumPy de pixels.
- Pour un **signal audio**, Librosa fournit un tableau d'échantillons et sa fréquence d'échantillonnage.
- Pour un **fichier ou segment audio**, Pydub fournit un objet `AudioSegment` facile à découper, assembler et exporter.

8.2 Bonnes Pratiques Générales

1. Toujours vérifier qu'un fichier a été correctement chargé.
2. Toujours inspecter la représentation obtenue : type, dimensions, durée, canaux, fréquence d'échantillonnage.
3. Distinguer le **fichier** sur disque de la **donnée décodée** en mémoire.
4. Conserver une copie des données originales avant toute modification.
5. Ne pas confondre BGR et RGB pour les images.
6. Ne pas rééchantillonner un audio sans savoir pourquoi.
7. Conserver explicitement les paramètres nécessaires à l'interprétation d'un signal.
8. Sauvegarder les résultats dans un répertoire distinct des données brutes.

8.3 Ce Qui Sera Traité Ultérieurement

Les rapports suivants pourront approfondir séparément :

- **Traitement d'image** : espaces colorimétriques, filtrage, histogrammes, seuillage, morphologie, contours, transformations géométriques et segmentation ;
- **Traitement audio** : prétraitement, tramage, fenêtrage, domaine fréquentiel, STFT, spectrogrammes, Mel, MFCC, F0 et autres descripteurs ;
- **Préparation pour l'apprentissage automatique** : normalisation, extraction de caractéristiques, constitution des tenseurs et préparation des jeux de données.

Conseil pour les débutants : avant de chercher à appliquer un traitement sophistiqué, assurez-vous de pouvoir expliquer exactement ce que contient la variable Python que vous manipulez : sa forme, son type, ses unités et la signification de chacune de ses dimensions.

Chapter 9

Aide-Mémoire NumPy et Matplotlib

9.1 NumPy

Besoin	Commande essentielle
Créer un tableau	<code>np.array(...)</code>
Connaître la forme	<code>a.shape</code>
Nombre de dimensions	<code>a.ndim</code>
Nombre total de valeurs	<code>a.size</code>
Type des valeurs	<code>a.dtype</code>
Sélectionner une tranche	<code>a[debut:fin]</code>
Région d'une image	<code>image[y1:y2, x1:x2]</code>
Copier	<code>a.copy()</code>
Changer de type	<code>a.astype(np.float32)</code>
Créer des indices	<code>np.arange(...)</code>
Valeurs régulièrement espacées	<code>np.linspace(...)</code>
Restructurer	<code>a.reshape(...)</code>
Concaténer	<code>np.concatenate([...])</code>
Minimum / maximum	<code>a.min(), a.max()</code>
Moyenne / écart-type	<code>a.mean(), a.std()</code>

9.2 Matplotlib

Besoin	Commande essentielle
Créer figure et axes	<code>fig, ax = plt.subplots()</code>
Tracer une courbe	<code>ax.plot(x, y)</code>
Afficher une image	<code>ax.imshow(image)</code>
Niveaux de gris	<code>ax.imshow(image, cmap="gray")</code>
Titre	<code>ax.set_title(...)</code>
Nommer l'axe horizontal	<code>ax.set_xlabel(...)</code>
Nommer l'axe vertical	<code>ax.set_ylabel(...)</code>
Légende	<code>ax.legend()</code>
Grille	<code>ax.grid(True)</code>
Masquer les axes d'une image	<code>ax.axis("off")</code>
Afficher	<code>plt.show()</code>
Sauvegarder	<code>fig.savefig(...)</code>

Chapter 10

Bibliographie et Ressources

10.1 Documentation Officielle

Outil	URL	Description
OpenCV	docs.opencv.org	Lecture, représentation, affichage et écriture d'images
NumPy	numpy.org/doc	Tableaux numériques multidimensionnels
Matplotlib	matplotlib.org/stable	Visualisation scientifique
Librosa	librosa.org/doc/latest	Chargement et représentation des signaux audio
Pydub	github.com/jiaaro/pydub	Manipulation de segments et fichiers audio
FFmpeg	ffmpeg.org	Codecs et conversion multimédia utilisés notamment avec Pydub

10.2 Fonctions à Maîtriser dans ce Rapport

Bibliothèque	Fonctions / objets	Finalité
OpenCV	<code>imread</code> , <code>imshow</code> , <code>imwrite</code> , <code>cvtColor</code> , <code>resize</code> , <code>flip</code>	ouvrir, afficher, représenter, manipuler et sauvegarder une image
Librosa	<code>load</code> , <code>display.waveshow</code>	ouvrir un audio, obtenir ses échantillons et afficher la forme d'onde
Pydub	<code>AudioSegment</code> , <code>from_file</code> , <code>slicing</code> , <code>export</code>	manipuler un fichier/segment audio
NumPy	<code>array</code> , <code>shape</code> , <code>ndim</code> , <code>size</code> , <code>dtype</code> , <code>slicing</code> , <code>arange</code> , <code>linspace</code> , <code>astype</code> , <code>reshape</code> , <code>concatenate</code>	comprendre et manipuler les tableaux numériques
Matplotlib	<code>subplots</code> , <code>plot</code> , <code>imshow</code> , <code>show</code> , <code>savefig</code>	visualiser images et signaux